

Syscall Heartbeat: A Decentralized Autonomous Task Orchestration Protocol

Version: 1.0

Network Target: MegaETH / EVM Compatible Chains

Date: October 2023

Abstract

Smart contracts on the Ethereum Virtual Machine (EVM) are passive entities; they cannot self-execute or wake up at scheduled intervals. They require an external "kick"—a transaction initiated by an Externally Owned Account (EOA)—to trigger logic.

Syscall Heartbeat is a decentralized, permissionless infrastructure designed to solve this limitation. It creates an incentivized market where independent actors ("Runners") compete to execute scheduled tasks for users in exchange for an algorithmic reward. By combining **EIP-1167 Minimal Proxies** for scalability, a **Gas-Metering Reimbursement Model** for economic viability, and an **Auto-Pruning Mechanism** for state hygiene, Syscall Heartbeat provides a "Set and Forget" automation layer for the MegaETH ecosystem.

1. Introduction

1.1 The Problem: The Passive Ledger

Blockchains are deterministic state machines. While they are excellent at verifying transactions, they lack an internal clock that can trigger code execution. A user who wants to harvest yield, update an oracle, or send a subscription payment every 24 hours must either:

1. Do it manually (prone to human error/forgetfulness).
2. Trust a centralized server with their private key (security risk).
3. Use complex decentralized networks that often require holding a specific utility token.

1.2 The Solution: Syscall Heartbeat

Syscall Heartbeat abstracts the complexity of automation. It allows any user to deploy a self-contained "Job Contract" funded with ETH. This contract holds the instructions for the task (Target address + Calldata) and the frequency (Interval).

The protocol relies on a network of third-party **Runners** (Bots). These Runners monitor the network, detect when a job is due, execute it, and are immediately reimbursed by the Job Contract with a profit margin.

2. System Architecture

The protocol utilizes a Hub-and-Spoke architecture to minimize gas costs and segregate user risk.

2.1 The Components

1. SyscallHeartbeatFactory (The Hub):

- Acts as the central registry for all active jobs.
- Manages the deployment of new user robots via **Clones**.
- Collects a small protocol fee (1%) on executions.
- Enforces security policies (e.g., minimum initial deposits).

2. SyscallHeartbeatJob (The Spoke/Robot):

- A sovereign smart contract owned by the user.
- Holds the user's ETH credits.
- Contains the specific logic: Target Address, Payload Data, and Time Interval.
- Autonomously calculates execution costs and pays the Runner.

3. The Runner (The Off-Chain Actor):

- Any EOA wallet running a script (RunBot).
- Monitors the Factory registry.
- Simulates transactions off-chain.
- Executes tasks to earn the reimbursement + 1% net profit.

3. Technical Implementation

3.1 EIP-1167: Minimal Proxy Clones

To ensure the protocol is affordable, we do not deploy the full contract logic for every user. Instead, we use the **EIP-1167 standard**.

- **The Master Implementation:** Deployed once. Contains all logic.

- **The Clone:** A tiny bytecode contract that simply delegates calls to the Master.
- **Benefit:** Deploying a robot costs a fraction of a standard deployment (approx. 1/10th of the gas).

3.2 Precise On-Chain Gas Metering

The core innovation of Syscall Heartbeat is its ability to calculate the cost of its own execution dynamically. The `executeTask` function operates as follows:

1. **Snapshot:** Records `startGas = gasleft()`.
2. **Execution:** Performs the user's requested call.
3. **Calculation:**

$GasUsed = startGas - gasleft()$

$TotalGas = GasUsed + Overhead$

Where $Overhead$ is a fixed constant (45,000 gas) covering the base transaction cost (21,000), ETH transfers, and logic execution.

4. **Billing:**

$Bill = TotalGas \times tx.gasprice$

3.3 The "Zombie Killer" Auto-Pruning Mechanism

A major risk in automation registries is "State Bloat"—thousands of empty contracts that bots must check, wasting CPU cycles.

Syscall Heartbeat implements a strict **Solvency Threshold** (\$0.002 ETH).

If a Job pays for an execution and its remaining balance falls below this threshold:

1. The Job calls the Factory to **deregister itself**.
2. The Factory uses a **Swap-and-Pop** algorithm ($O(1)$ complexity) to remove the Job from the active array.
3. The Job returns any "dust" (remaining wei) to the owner.
4. The Job pays the Runner with its final balance.

Result: The registry only ever contains solvent, active jobs.

4. Economic Model

The system is designed to be a self-sustaining open market.

4.1 Fee Structure

Every successful execution incurs a cost calculated as:

$$TotalCost = GasCost \times 1.02$$

This 2% markup is split as follows:

- **101% to the Runner:** The Runner is reimbursed 100% of their gas costs plus a **1% Net Profit**. This ensures that as long as gas prices are stable, running a bot is always profitable.
- **1% to the Factory:** A protocol fee collected by the Factory owner to sustain development.

4.2 Incentives

- **For Users:** No need to run infrastructure. They pay a negligible premium (2%) for reliability.
 - **For Runners:** Risk-free profit. The transaction will revert if the Job cannot pay, meaning the Runner never executes a losing transaction (provided they simulate off-chain).
-

5. Security & Threat Model

The protocol has been hardened against common DeFi attack vectors.

5.1 Appropriation & Ownership Theft

Threat: An attacker tries to "hijack" a newly created clone.

Mitigation:

- **Atomic Initialization:** The Factory creates the clone and calls `initialize()` in the *same transaction*. There is no gap block where the contract exists uninitialized.
- **Initialization Lock:** The `initialize()` function has a `require(!initialized)` guard. It can only be called once.

5.2 Reentrancy Attacks

Threat: The User's target contract calls back into the Job to drain funds.

Mitigation:

- **Checks-Effects-Interactions (CEI):** The state variable `lastRun` is updated *before* the external call is made.

Solidity

```
lastRun = block.timestamp; // Effect
```

(bool success,) = target.call(data); // Interaction

If the target calls back, the time check require(now >= lastRun + interval) will fail immediately.

5.3 Registry Pollution (DoS)

Threat: An attacker spams the Factory with 100,000 insolvent contracts to slow down Runners.

Mitigation:

- **Minimum Deposit:** The createJob function requires a msg.value >= MIN_INITIAL_DEPOSIT (0.002 ETH). Creating spam becomes economically prohibitive.
- **O(1) Deletion:** Removing a job costs the same gas amount regardless of registry size, preventing gas-limit DOS attacks during cleanup.

5.4 Insolvency / Griefing

Threat: A user provides a task that consumes infinite gas, causing the Runner to run out of gas and fail.

Mitigation:

- **RunBot Simulation:** Runners are expected to use eth_estimateGas before broadcasting. If a user's task reverts or consumes too much gas, the simulation fails off-chain, and the Runner does not broadcast the transaction. The User simply hurts themselves (their job is never run).

6. Use Cases

1. **DCA (Dollar Cost Averaging):** A user deploys a job calling a DEX router to swap 100 USDC for ETH every 7 days.
2. **Payroll:** A DAO deploys a job to release vesting tokens to contributors every month.
3. **Keepers for Protocols:** A DeFi protocol deploys jobs to trigger liquidations or update Oracle prices regularly.
4. **Heartbeat Monitoring:** Using the event emission as a proof-of-liveness for other systems.

7. Conclusion

Syscall Heartbeat represents a robust, minimal, and secure approach to blockchain automation. By aligning economic incentives between Users (who pay for service) and Runners (who execute for profit), and by strictly managing state hygiene via auto-pruning, the protocol ensures high reliability without centralization.

It transforms the passive MegaETH ledger into an active, event-driven ecosystem.